FetalAI

Using Machine Learning To Predict And Monitor Fetal Health

A Cardiotocography (CTG) Data Analysis & Classification Web Application

Project Title **FetalAI: Using Machine Learning To Predict And Monitor Fetal Health**	**Document Type** **Final Report**
Domain Healthcare • Prenatal Monitoring • Machine Learning	**Technology Stack** • Python • Scikit-learn • Flask • HTML/CSS/JavaScript
ML Model Random Forest Classifier	**Dataset Features** 21 Numeric CTG Features
Output Classes Normal • Suspect • Pathological	**Document Status** Final Report

1. Introduction

Fetal health monitoring is a critical aspect of obstetric care. Cardiotocography (CTG) is one of the most widely used techniques for evaluating fetal well-being during pregnancy and labour. It records the fetal heart rate (FHR) and uterine contractions simultaneously, allowing clinicians to detect early signs of fetal distress. However, manual interpretation of CTG data can be subjective and time-consuming.

This project addresses that challenge by developing an automated Fetal Health Classification System. Using a supervised machine learning approach, the system classifies fetal health into three categories — Normal, Suspect, and Pathological — based on 21 clinically relevant CTG features. The solution is deployed as a browser-accessible Flask web application, bridging the gap between data science and practical clinical decision support.

The project was undertaken as part of a course requirement and aims to demonstrate an end-to-end machine learning pipeline, from raw data ingestion to model deployment within a user-facing interface.

2. Project Objectives

The key objectives of this project were:

- To study and preprocess the UCI Fetal Health dataset, which consists of 2,126 CTG records with 21 numeric features.
- To train a machine learning classifier capable of distinguishing between Normal, Suspect, and Pathological fetal health states.
- To integrate the trained model into a Flask-based web application with a clean, user-friendly interface.
- To implement proper input validation, error handling, and confidence reporting in the prediction workflow.
- To produce comprehensive documentation and present the project in a manner suitable for academic review.

3. System Architecture and Components

The project is divided into three major components: the web application layer, the machine learning model, and the supporting dataset. The following table summarises each component:

#	Component	Description
1	**Web Application**	A Flask-based server (app.py) hosting multiple routes including a home page, prediction form, result view, contact page, and model information page. The app also exposes a RESTful JSON API endpoint and a health-check route.
2	**ML Model**	A Random Forest classifier trained on 21 CTG features extracted from the dataset. The trained model is serialised using Python's pickle library and stored as optimized_fetal_health_model.pkl. It is loaded once at startup and used for all predictions.
3	**Dataset**	The UCI Fetal Health dataset (fetal_health.csv) containing 2,126 records. Each record includes 21 numeric CTG features and a label indicating the fetal health class (1 = Normal, 2 = Suspect, 3 = Pathological).

3.1 Web Application

The web layer is implemented using the Flask microframework in Python. The application defines several routes, each serving a specific purpose. The home page (/) provides context about the project and directs users to the prediction tool. The prediction route (/predict) renders a structured HTML form containing 21 labelled input fields, each corresponding to a CTG feature. Upon form submission, the backend collects the inputs, validates them, converts the values to floating-point numbers, and passes them to the loaded model for inference.

The result page displays the predicted health category alongside a confidence percentage derived from the model's probability output. A contact page and a model-info page were additionally implemented to round out the application and demonstrate awareness of good software design practices.

3.2 Machine Learning Model

A Random Forest classifier was selected for this project due to its robustness, interpretability relative to deep learning approaches, and its ability to handle multi-class classification without significant hyperparameter tuning. The model was trained using the scikit-learn library in Python.

During training, the 21 CTG features were used directly as inputs without dimensionality reduction, preserving all available clinical information. The dataset was split into training and testing subsets using an 80/20 ratio. Model performance was evaluated on the test set, and the best-performing configuration was exported as a .pkl file.

3.3 Dataset

The dataset used in this project is the publicly available UCI Fetal Health dataset. It contains 2,126 CTG examination records, each described by 21 numeric attributes including baseline fetal heart rate, accelerations, fetal movements, uterine contractions, and various histogram statistics derived from CTG signals. The target variable indicates one of three fetal health states: Normal (class 1), Suspect (class 2), and Pathological (class 3).

The dataset exhibits a class imbalance, with the Normal class being significantly more prevalent than the Suspect and Pathological classes. This imbalance was taken into account during model evaluation, and performance metrics were examined per class to avoid misleadingly high aggregate accuracy scores.

4. User Experience and Interface Design

Considerable attention was given to making the application accessible and easy to navigate for non-technical users. The home page introduces the project and provides clear guidance on how to use the prediction tool. The prediction form is laid out in a structured, readable manner with all 21 fields clearly labelled.

After submission, the result page presents the predicted fetal health category in plain language, along with a confidence level expressed as a percentage. Contextual guidance is also displayed to

help users interpret the result appropriately — particularly important given the clinical nature of the predictions.

Error handling was implemented to catch missing or non-numeric inputs, displaying informative messages that guide users to correct their entries without losing previously entered data.

5. Technical Implementation Details

The technical stack used in this project consists of the following:

- Backend: Python 3 with Flask for routing and request handling, and scikit-learn for model training and inference.
- Frontend: HTML5 templates managed via Flask's Jinja2 templating engine, styled with CSS for a clean presentation.
- Model Serialisation: The trained Random Forest model was saved using Python's pickle module for efficient loading at runtime.
- API: A JSON endpoint was implemented to support programmatic access and automated testing independent of the web UI.

One technically important consideration was ensuring that the order of input features was consistent across three separate points: the order in which features appear in the training dataset, the order in which the model expects input at inference time, and the order in which the web form collects and sends user input. Any mismatch in this ordering would produce incorrect predictions without raising an explicit error, making this a non-obvious but critical alignment requirement.

6. Results and Outcomes

The trained Random Forest model demonstrated satisfactory performance on the test dataset. Given the class imbalance in the data, performance was assessed across all three categories rather than relying solely on overall accuracy.

The application successfully classifies fetal health for all valid input combinations and returns a prediction along with the associated confidence score. The system is intentionally framed as a decision-support demonstration rather than a clinical diagnostic tool, and this distinction is made clear throughout the application’s interface and documentation.

The project deliverables include the working web application, the trained model file, the source dataset, all Python scripts, HTML templates, and this report — together forming a complete, submission-ready package.

7. Challenges Encountered and Lessons Learned

Several challenges were encountered during the development of this project:

7.1 Feature Order Consistency

Maintaining a consistent feature order between the dataset, model training, and web form was one of the most subtle yet consequential challenges. The order had to be explicitly verified and locked at each stage to avoid silent prediction errors. This was resolved by defining a single master feature list that was referenced throughout the codebase.

7.2 Class Imbalance

The dataset's class imbalance — where the Normal class dominates over the Suspect and Pathological classes — posed a risk of biased evaluation. This was handled by examining class-wise performance metrics (precision, recall, and F1-score for each class) rather than relying solely on accuracy. Future work could explore techniques such as SMOTE oversampling or class-weighted training to further mitigate this.

7.3 Full-Stack Integration

Bridging the gap between a data science pipeline and a functional web application required learning and applying Flask routing, Jinja2 templating, and HTML form handling alongside the machine learning components. This multi-disciplinary requirement was challenging but provided meaningful practical experience in full-stack development.

8. Suggestions for Future Enhancement

While the current implementation satisfies the project requirements, the following enhancements are proposed for future iterations:

- Input Validation and Units: Adding per-field validation rules with acceptable ranges and measurement units (e.g., beats per minute for baseline FHR) would reduce user error and improve data quality.

- Visual Analytics: Incorporating charts such as feature importance plots, confusion matrices, and ROC curves would strengthen the analytical aspect of the application and provide better insight into model behaviour.
- Batch Prediction via CSV Upload: Allowing users to upload a CSV file containing multiple CTG records for batch classification would significantly improve usability in data-heavy scenarios.
- Cloud Deployment: Hosting the application on a cloud platform (such as Heroku, Render, or AWS Elastic Beanstalk) would make it publicly accessible and demonstrate deployment skills beyond local development.
- Model Explainability: Integrating SHAP (SHapley Additive exPlanations) values to highlight the top contributing features for each individual prediction would improve transparency and clinical interpretability.

9. Conclusion

This project successfully demonstrates the design and implementation of a machine learning-based fetal health classification system, from raw CTG data to a deployed web application. The work covers the full pipeline: data preprocessing, model training, evaluation, integration, and user interface development.

The Random Forest model provides a reliable multi-class classification capability, and the Flask application makes it accessible to users without requiring any programming knowledge. The project is structured to fulfil the academic submission requirements and reflects an understanding of both the technical and communicative aspects of applied data science.

Working on this project reinforced the importance of end-to-end thinking in software development — where individual components such as the model, the data pipeline, and the web interface must work together seamlessly. It also highlighted the value of clear documentation and user-centred design, even in an academic context.

References

[1] Ayres-de-Campos, D., Bernardes, J., Garrido, A., Marques-de-Sá, J., & Pereira-Leite, L. (2000). SisPorto 2.0: A program for automated analysis of cardiotocograms. Journal of Maternal-Fetal Medicine, 9(5), 311–318.

[2] UCI Machine Learning Repository. (2020). Fetal Health Classification Dataset. Retrieved from https://www.kaggle.com/andrewmvd/fetal-health-classification

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Flask Documentation. (2024). Flask — A lightweight WSGI web application framework. https://flask.palletsprojects.com

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.